SPOTY

Estacione com inteligência

Backlog & Relatório do Projeto

O que já está pronto, o que falta, o que ainda não é possível — e as próximas ideias

Concluído | Pendente | Não conseguimos | Ideias

Versão 1.0 • Setembro de 2026

Documento interno de planejamento — não distribuir externamente

Preparado a partir dos arquivos de código-fonte e das notas do time, em setembro de 2026

Sumário

1 Visão geral do projeto

2 O que já está concluído

3 O que está pendente

4 O que não conseguimos fazer (por ora)

5 Sistema de cadastro — duas vertentes

6 Possíveis ideias — monetização e crescimento

7 Banco de dados — decisões e schema

8 Resumo executivo do backlog

1 Visão geral do projeto

O Spoty é uma plataforma (site + futuro app) que ajuda motoristas a encontrar vagas de estacionamento — rotativas, privativas ou mensais — perto de onde estão, e ajuda donos de estacionamento a ganhar visibilidade e novos clientes. Este documento organiza o backlog do projeto com base no que já foi construído até agora (site institucional, páginas, estilos e scripts) e nas decisões e observações levantadas pelo time.

O conteúdo está dividido em quatro frentes — **concluído**, **pendente**, **o que não conseguimos fazer no momento** e **possíveis ideias futuras** — além de uma seção dedicada ao sistema de cadastro (que terá duas vertentes de usuário) e um resumo executivo final para consulta rápida.

2 O que já está concluído

Itens já implementados no front-end do site, com base no código-fonte atual (HTML, CSS e JavaScript).

Item	Detalhes	Status
Página inicial (index.html)	Seção hero com busca por endereço, botão de geolocalização, mapa interativo, depoimentos de usuários e seção “Como funciona”.	Concluído
Mapa interativo real	Integração com Leaflet + OpenStreetMap, restrito ao território brasileiro, com marcador da posição do usuário.	Concluído
Busca de endereço real	Conversão de endereço digitado em coordenadas via API Nominatim (OpenStreetMap), com recentralização automática do mapa.	Concluído

Item	Detalhes	Status
Geolocalização do usuário	Uso da Geolocation API do navegador para centralizar o mapa na posição real do usuário, com tratamento de erros (permissão negada, indisponível, tempo excedido).	Concluído
Identidade visual	Logotipo, paleta de cores (azul, verde, amarelo) e estilo visual (style.css) aplicados em todas as páginas.	Concluído
Página de cadastro (front-end)	Formulário único com nome, e-mail, telefone, senha, confirmação de senha e aceite de termos.	Concluído (parcial)
Página “Para empresas”	Página institucional voltada a donos de estacionamento (empresas.html).	Concluído
Central de ajuda	Página de perguntas frequentes / suporte (central-ajuda.html).	Concluído
Fale conosco	Página e script de formulário de contato (fale-conosco.html / fale-conosco.js).	Concluído (front)
Termos de uso e Política de privacidade — rascunho	Textos já redigidos e publicados nas páginas termos-uso.html e privacidade.html, cobrindo uso do serviço, cadastro, pagamentos, responsabilidades e propriedade intelectual.	Rascunho / a validar
Transições de página e notificações (toast)	Efeito de transição entre páginas e sistema de mensagens temporárias (paginas.js / script.js).	Concluído
Selos de loja (App Store / Google Play)	Artes prontas (badge-appstore.svg, badge-googleplay.svg), hoje apontando para links genéricos das lojas — o app ainda não existe.	Arte pronta / app pendente

Observação: os itens de termos/privacidade e selos de loja aparecem aqui porque já existe material pronto no site, mas — como o time sinalizou — ainda precisam de validação/continuidade e por isso também são tratados como pendências na próxima seção.

3 O que está pendente

Itens levantados pelo time como pendentes, essenciais para o Spoty sair do front-end estático e virar um produto funcional de verdade.

Item	Detalhes	Status
Banco de dados	Modelagem e criação do banco de dados do projeto (usuários, estacionamentos, vagas, reservas, pagamentos).	Pendente
Back-end para o banco de dados	API/servidor que conecta o site/app ao banco de dados, hoje inexistente — todo o mapa de vagas é gerado no próprio navegador.	Pendente

Item	Detalhes	Status
Termos de uso e Política de privacidade — validação final	Revisão jurídica/definitiva dos textos já rascunhados, antes de valerem como termos oficiais.	Pendente
Cadastro em duas vertentes	Hoje existe apenas um formulário genérico de cadastro; falta separar o fluxo entre “dono de estacionamento” e “cliente que vai estacionar” (detalhado na seção 5).	Pendente
Autenticação e login	Criação de conta hoje não persiste em lugar nenhum (depende do banco de dados/back-end); login (“Entrar”) ainda é apenas um link sem função.	Pendente
Pagamentos reais	Cartão de crédito, PIX e “saldo Spoty” são mencionados nos Termos de Uso, mas ainda não há integração de pagamento implementada.	Pendente
Painel do estabelecimento	Interface para o dono controlar: nome do comércio, quantidade de vagas existentes, vagas ocupadas e livres, valor da vaga por 30 min / 1 hora / diária, e controle de entrada e saída de carros por placa (ver ideia detalhada na seção 6). Já especificado pelo time; implementação ainda não iniciada.	Pendente
Aplicativo mobile	Os selos de loja já existem na página, mas o aplicativo em si (iOS/Android) ainda não foi desenvolvido.	Pendente

4 O que não conseguimos fazer (por ora)

Limitação técnica identificada pelo time: sem uma fonte de dados real de ocupação, não é possível mostrar a disponibilidade real das vagas.

Item	Detalhes	Status
Detecção real de vagas livres	Não temos satélites nem sensores instalados nas ruas ou estacionamentos — por isso, hoje não há como saber, de fato, quais carros estão parados ou quais vagas estão livres em um determinado local.	Não conseguimos
Vagas exibidas no mapa hoje	Atualmente, o número de vagas e os marcadores exibidos no mapa da página inicial são gerados aleatoriamente pelo script (script.js), apenas como demonstração visual — não refletem a ocupação real.	Simulado

Caminhos possíveis para resolver isso no futuro:

- Parceria direta com os donos de estacionamento, que informam manualmente a ocupação pelo painel de controle (ver seção 6) — solução mais viável a curto prazo, sem depender de hardware.
- Sensores de vaga (IoT) instalados em estacionamentos parceiros, com custo de instalação.

- Câmeras com reconhecimento de imagem para contar vagas ocupadas/livres.
- Integração com dados públicos de mobilidade urbana, quando disponíveis na cidade.

Sem uma dessas soluções, o mapa continuará mostrando uma estimativa simulada, e não a disponibilidade real — isso deve ficar claro para o usuário até que o problema seja resolvido.

5 Sistema de cadastro — duas vertentes

O cadastro do Spoty precisa reconhecer dois perfis de usuário diferentes, com jornadas e dados distintos. Hoje (cadastro.html) existe apenas um formulário único e genérico — este item está pendente de divisão.

Dono de estacionamento / estabelecimento	Cliente que vai estacionar
• Cadastra o estabelecimento (nome, endereço, horário de funcionamento) • Define preço por hora / mensalidade • Acessa o futuro painel de controle de vagas • Paga assinatura mensal pelo uso da plataforma (ver seção 6)	• Cadastro simples para buscar e localizar vagas • Recebe recomendações de estacionamentos próximos • Realiza pagamento das vagas utilizadas • Pode assistir anúncios como parte do modelo de receita (ver seção 6)

Atualização: no desenho do banco de dados (seção 7.2), ficou definido que essas duas vertentes não são contas separadas nem exclusivas — a mesma pessoa pode ser dona de um estabelecimento e cliente em outro. As jornadas acima continuam válidas como fluxos de uso, mas tecnicamente serão uma conta única, com o papel identificado pelos dados (possuir um estabelecimento cadastrado, ter feito reservas), e não por um campo fixo escolhido no cadastro.

6 Possíveis ideias — monetização e crescimento

Ideias levantadas pelo time para gerar receita e engajamento, ainda não implementadas — registradas aqui para avaliação e priorização futura.

Ideia 1 — Assinatura mensal para donos de estacionamento

Donos de estacionamento pagam um valor mensal (“valor X”) para ter acesso a uma interface de controle do próprio estacionamento, com:

- Nome do comércio;
- Quantidade total de vagas existentes;
- Vagas ocupadas e vagas livres, atualizadas em tempo real;
- Valor da vaga por 30 minutos, por 1 hora e por diária;
- Horário de funcionamento;
- Controle de entrada e saída de veículos apenas pela placa do carro;
- Recomendação automática do estabelecimento para clientes finais próximos (o cliente recebe um aviso do tipo “há um estacionamento perto de você”).

Em troca do valor pago, o estabelecimento ganha mais visibilidade/popularidade dentro do aplicativo, e o Spoty ganha uma fonte de receita recorrente. Essa interface já foi especificada pelo time — os detalhes de campos e comportamento estão na tabela de pendências (seção 3) e a implementação segue as tabelas `estabelecimentos` e `vagas` definidas na seção 7.

Ideia 2 — Anúncios para o cliente final

Clientes que vão estacionar podem assistir anúncios dentro do app/site; em troca, o Spoty (os desenvolvedores) recebe receita apenas pela utilização da plataforma, sem custo direto para o motorista.

Ideia 3 — Combinação dos dois modelos

As duas ideias acima se complementam: donos de estabelecimento pagam pela ferramenta de gestão e visibilidade, enquanto o público final gera receita por meio de anúncios — dois fluxos de monetização distintos para as duas vertentes de usuário descritas na seção 5.

7 Banco de dados — decisões e schema

Registro das decisões técnicas tomadas sobre o banco de dados do Spoty e o script SQL final, já revisado, pronto para ser executado no MySQL Workbench.

7.1 Decisões técnicas

- Sistema de banco de dados: **MySQL** (opção considerada e descartada: Supabase/Postgres).
- Ferramenta de administração: **MySQL Workbench**.
- Linguagem do back-end: **Python** (framework ainda a definir — Flask, FastAPI ou Django).
- Senhas nunca são salvas em texto puro — sempre com hash (ex: bcrypt) antes de gravar no banco.

7.2 Correção de modelagem — papel do usuário

Foi identificado que o mesmo usuário pode ser, ao mesmo tempo, dono de um estabelecimento e cliente procurando vaga em outro lugar. Por isso, o campo fixo “tipo” (cliente/dono) foi removido da tabela `usuarios`. Cada conta é única, e o papel da pessoa passa a ser deduzido pelos próprios dados:

- É dono → existe uma linha em `estabelecimentos` com esse `dono_id`;
- É cliente → existe uma linha em `reservas` com esse `usuario_id`.

Na prática, isso também simplifica o cadastro: em vez de dois formulários totalmente separados, o ideal passa a ser um cadastro único (nome, e-mail, senha), com a opção de “cadastrar meu estacionamento” disponível a qualquer momento dentro do app — usando a mesma conta.

7.3 Schema final (SQL)

Script completo, dividido por tabela para facilitar a leitura. A ordem de criação já respeita as dependências entre chaves estrangeiras.

Banco de dados e tabela “usuarios”

```
CREATE DATABASE IF NOT EXISTS spoty
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;

USE spoty;

-- Conta unica: qualquer usuario pode ser dono e/ou cliente
CREATE TABLE usuarios (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nome VARCHAR(120) NOT NULL,
  email VARCHAR(150) NOT NULL UNIQUE,
  senha VARCHAR(255) NOT NULL,
  telefone VARCHAR(20),
  criado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;
```

Tabela “estabelecimentos”

```
CREATE TABLE estabelecimentos (
  id INT AUTO_INCREMENT PRIMARY KEY,
  dono_id INT NOT NULL,
  nome VARCHAR(150) NOT NULL,
  endereco VARCHAR(255) NOT NULL,
  latitude DECIMAL(10,8),
  longitude DECIMAL(11,8),
  preco_hora DECIMAL(6,2) NOT NULL CHECK (preco_hora >= 0),
  horario_abertura TIME,
  horario_fechamento TIME,
  criado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (dono_id) REFERENCES usuarios(id) ON DELETE RESTRICT,
  INDEX idx_estabelecimentos_dono (dono_id)
) ENGINE=InnoDB;
```

Tabela “vagas”

```
CREATE TABLE vagas (
  id INT AUTO_INCREMENT PRIMARY KEY,
  estabelecimento_id INT NOT NULL,
  codigo VARCHAR(20),
  status ENUM('livre','ocupada') NOT NULL DEFAULT 'livre',
  atualizado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    ON UPDATE CURRENT_TIMESTAMP,
  FOREIGN KEY (estabelecimento_id) REFERENCES estabelecimentos(id)
    ON DELETE CASCADE,
  UNIQUE KEY uk_vaga_por_estabelecimento (estabelecimento_id, codigo),
  INDEX idx_vagas_estab_status (estabelecimento_id, status)
) ENGINE=InnoDB;
```

Tabela “reservas”

```
CREATE TABLE reservas (
  id INT AUTO_INCREMENT PRIMARY KEY,
  usuario_id INT NOT NULL,
  vaga_id INT NOT NULL,
  entrada DATETIME NOT NULL,
  saida DATETIME,
  valor_total DECIMAL(8,2),
  status ENUM('em_andamento','concluida','cancelada')
    NOT NULL DEFAULT 'em_andamento',
  FOREIGN KEY (usuario_id) REFERENCES usuarios(id) ON DELETE RESTRICT,
  FOREIGN KEY (vaga_id) REFERENCES vagas(id) ON DELETE RESTRICT,
  INDEX idx_reservas_usuario (usuario_id),
  INDEX idx_reservas_vaga (vaga_id)
) ENGINE=InnoDB;
```

Tabela “pagamentos”

```
CREATE TABLE pagamentos (
  id INT AUTO_INCREMENT PRIMARY KEY,
  reserva_id INT NOT NULL,
  metodo ENUM('cartao','pix','saldo') NOT NULL,
  valor DECIMAL(8,2) NOT NULL CHECK (valor >= 0),
  status ENUM('pendente','aprovado','recusado')
    NOT NULL DEFAULT 'pendente',
  criado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (reserva_id) REFERENCES reservas(id) ON DELETE RESTRICT,
  INDEX idx_pagamentos_reserva (reserva_id)
) ENGINE=InnoDB;
```

Tabela “assinaturas”

```
CREATE TABLE assinaturas (
  id INT AUTO_INCREMENT PRIMARY KEY,
  estabelecimento_id INT NOT NULL,
  plano VARCHAR(50) NOT NULL,
  valor_mensal DECIMAL(6,2) NOT NULL CHECK (valor_mensal >= 0),
  status ENUM('ativa','cancelada','atrasada') NOT NULL DEFAULT 'ativa',
  inicio DATE NOT NULL,
  fim DATE,
  FOREIGN KEY (estabelecimento_id) REFERENCES estabelecimentos(id)
    ON DELETE CASCADE,
  INDEX idx_assinaturas_estab (estabelecimento_id)
) ENGINE=InnoDB;
```

7.4 O que foi revisado nesse schema, e por quê

- **utf8mb4** no banco inteiro — evita quebra com acentos, “ç” ou emoji em nomes e comentários.
- **ENGINE=InnoDB** explícito — garante chave estrangeira e transações (importante quando reserva e pagamento precisam ser tratados juntos).
- **ON DELETE RESTRICT** em usuários → estabelecimentos/reservas e reservas → pagamentos — impede apagar um usuário ou reserva que tenha histórico ligado, protegendo dados financeiros.
- **ON DELETE CASCADE** em estabelecimentos → vagas/assinaturas — se o estabelecimento for removido, suas vagas e assinatura são removidas junto, como esperado.
- **UNIQUE (estabelecimento_id, codigo)** em vagas — impede duas vagas com o mesmo código dentro do mesmo estacionamento.
- **CHECK** em valores monetários — impede preço ou pagamento negativo (MySQL 8.0.16+).
- Índices extras em `dono_id`, `estabelecimento_id + status`, `usuario_id`, `vaga_id` e `reserva_id` — deixam rápidas as buscas mais comuns do app (vagas livres de um estacionamento, reservas de um usuário, pagamento de uma reserva).
- Campo `tipo` removido de usuários — ver correção de modelagem em 7.2.

7.5 Capacidade estimada

O `id INT AUTO_INCREMENT` comporta até aproximadamente 2,1 bilhões de linhas por tabela — bem acima do que este projeto deve alcançar. O limite real, na prática, é o servidor onde o banco roda, não a estrutura das tabelas:

- Hospedagem compartilhada básica: dezenas de milhares de usuários cadastrados, algumas centenas em uso simultâneo;
- VPS pequeno/médio (2-4 GB RAM): centenas de milhares de usuários, poucos milhares em uso simultâneo — patamar confortável para a fase atual do Spoty;
- Servidor dedicado / cloud maior: milhões de usuários, dezenas de milhares em uso simultâneo.

Se for necessário eliminar de vez esse teto (mais barato de ajustar agora do que depois), o caminho é trocar `INT` por `BIGINT` nos campos de id, elevando o limite para cerca de 9 quintilhões de linhas.

8 Resumo executivo do backlog

Visão consolidada de todos os itens deste documento, em ordem de prioridade sugerida.

Item	Detalhes	Status
Schema do banco de dados (MySQL)	Todas as tabelas desenhadas e revisadas (seção 7); falta criar fisicamente no MySQL Workbench e conectar ao back-end.	Modelagem concluída
Back-end em Python	Depende do banco de dados criado.	Pendente
Cadastro (conta única)	Sem separação rígida dono × cliente; papel deduzido pelos dados (seção 7.2).	Pendente
Login / autenticação	Depende do banco de dados e back-end.	Pendente
Termos de uso e privacidade	Rascunho pronto; falta validação final.	A validar
Painel do estabelecimento	Campos já especificados: nome, vagas totais/ocupadas/livres, preço 30min/1h/diária, entrada e saída por placa.	Pendente
Pagamentos (cartão / PIX / saldo)	Mencionado nos termos, não implementado.	Pendente
Vagas livres em tempo real	Sem satélites/sensores — hoje o mapa é simulado.	Não conseguimos
Site institucional (páginas, estilo, mapa)	Front-end funcional e navegável.	Concluído
Ideias de monetização	Assinatura para estabelecimentos + anúncios para clientes.	Ideia / a avaliar

Fim do documento. Este backlog deve ser revisado e atualizado conforme o projeto evolui.